UNIVERSITY OF LIVERPOOL

COMP530

Project 6: Temporal Path Explorer & Visualizer

Specification and Proposed Design

Prepared by Team 9:

Ce Wang
Bowen Li
Xiaohan Xu
Zixuan Tang
Wenbin Zeng
Yiqing Liu
Ankit Ranjan

February 16, 2026

1 Introduction

Many real-world systems can be represented as networks that change over time. Examples include a communication network with a scheduled transport system, a social network formed by time-constrained interactions, and information on disease-spreading processes. In these cases, time is an important factor in determining how influence and information propagate through the network.

Temporal networks extend graph models by associating time with the edges, supporting the study of time-dependent connectivity[1]. The core concept is the temporal path constraint, which is a sequence of edges in strictly increasing time order. Temporal paths form the basis of temporal connectivity, which determines when a node can be reached from a given source under temporal constraints. This concept is important for understanding mechanisms, like information diffusion, epidemic spreading, and network resistance.

The algorithmic foundations of temporal connectivity are well-studied, including simple and efficient approaches such as temporal breadth-first search (BFS)[2]. The existing graph visualisation tools, designed for static networks, provide limited support for exploring the temporal constraints and observing how connectivity evolves over time. This results in mismatch of temporal networks and tools that available to explore them.

The Temporal Path Explorer and Visualizer project aims to handle this gap with web-based tool to explore temporal paths with changing networks. The system will let users to upload time-based network data and observe how connectivity from a selected node source evolves over time. Linking the temporal connectivity algorithm with visualisation, this tool makes the concept of time-constrained paths explicit and concrete, that is enabling users to observe the impact of temporal ordering on network connectivity.

The core of the system will is a simple temporal connectivity algorithm, which will compute reachable nodes stepwise, such as a one-pass temporal BFS. The results are reflected immediately in the visual interface, facilitating users to follow the temporal expansion of reachable nodes. This merging of computation and visualisation supports exploratory analysis and helps to connect, that links the theoretical and practical understanding.

The distinctive feature is that users will be able to remove or delay certain temporal edges and observe how these changes will affect the network. This function of the network will draw attention towards the temporal processes to the events and enables users to experiment with problems, such as delayed communication or missing connections.

The project is primarily implementation-driven and follow a interactive software system. It also include research on time-varying networks, temporal path algorithms, data structures, and the visual encoding used in the system. The project follows most recent software development practices, with implementation guided by research and work.

This includes clear requirements analysis, careful system design, and the adoption of an appropriate software development life cycle. The project is carried out by a team, reflecting professional practice in which effective communication, task coordination, and documentation.

2 Description

Aim

The objective of this project is to design and develop an interactive web application titled 'Temporal Path Exploration and Visualisation Tool' enables users upload temporal network data using this tool to easily explore how paths form and change over time.

Objectives

- First, the application should/must support the processing of common data formats (such as CSV or JSON) including nodes, edges and timestamps
- Second. a temporal reachability algorithm will be implemented, BFS for instance, compute reachable nodes from a given source node in a way that is easy to understand.
- Third, the interface/UI is supposed to be interactive and user-friendly to facilitate the visualisation of the network, which will highlight the nodes and edges as they become reachable over time.
- Fourth, this app will include a 'hypothetical analysis' feature, which lets users interactively remove or delay specific time limits and immediately see the impact of these actions on the network's overall reachability. This shows how important it is to look closely at individual events in order to understand what is happening.

Milestones

1. The first milestone is the clear definition of temporal network standards and implementation of formatted data uploading and parsing.
2. The second milestone is the implementation and verification of the temporal reachability algorithm using small test cases.
3. The third stage involves integration of UI and the verified temporal reachability algorithm.
4. Following the third, add time control block/mechanism: slider, frame by frame progression.
5. The final phase of the project will be the implementation of the 'what-if' functionality, which will allow users to modify temporal edges and observe the impact on reachability.

Expected Outcomes

The outcome of this project is a fully functional web application developed using Streamlit within Python, featuring a clean and intuitive interface.

This project puts practical development first, while also doing some research. It involves looking at important research on networks and BFS algorithms, and using this knowledge to build a tool that is useful in practice and teaches the subject well.

3 Project Plan

3.1 Achieving Project Aims Methodologies

To ensure the successful outcome of the project Temporal Path Explorer & Visualizer, our team's methodology is built based on a SWOT analysis. This Methodology allows us to capture our internal capabilities and effectively addressing technical and environmental challenges.

Strengths

We have the based knowledge to process a clear algorithmic foundation, the one-pass temporal BFS. The basic framework of the project has been determined. And for the time management, the project is structured as group work, allowing for parallel development of the backend algorithm and the frontend interface.

Weaknesses

The challenge lies in our lack of practical experience in developing visualizations. On the other hand, the group members are not familiar with each other, which may result in the lack of communication within the team.

Opportunities

There is already a big number of materials available about the BFS algorithm. And our supervisor specializes in BFS algorithm development and can provide professional guidance in the field.

Threats

The main external challenge lies in the limited time. The complexity of the project introduces uncertainty and risks.

3.2 Criteria for Project Success

The success of this project will be evaluated a multi-dimensional framework. The details can be identified as:

Algorithmic Correctness: The project product must accurately identify all time-respecting paths. The output from the back terminal must comply with the calculation results through the one-pass temporal BFS algorithms.

Functional Completeness: The web tool must implement the visualization of algorithm path finding correctly. The final product is supposed to show the pathfinding animation in random charts. And the software is supposed to meet seamless file uploading and the operational "what-if" functionality for edge modification.

System Performance: The software must run without noticeable delay. Given the nature of dynamic networks, the tool must maintain high responsiveness. A key success metric is the "immediate" update of the visualization (under 500ms latency) when a user performs a "what-if" action.

Academic and Technical Compliance: Success also is supposed to meet the formal requirements of the University of Liverpool policy, including the submission of a high-quality demonstration and a comprehensive completion report by the stated deadlines in 2026.

3.3 Software Development Lifecycle: The Agile Approach

Considering the exploratory nature of temporal network visualization, our team will follow an Agile development life circle model, which allows for iterative refinement, which is critical when designing complex interactive tools.

Through Agile model as a systematic development process that can point to the understanding of the scope and complication of the total development process is essential to achieve the characteristics of a successful system.

The development process for this project will be strictly divided into the following five stages to ensure high-quality completion of the task before the deadline:

1. **Phase one: Requirements Analysis and Architecture Design (Weeks 1-3):** In this phase, the team will deepen its understanding of the one-pass temporal BFS theory and determine the front-end and back-end architecture of the system. Each small group will propose the interfaces required for its assigned section and refine the specifications of the data input interfaces, laying the foundation for the embedding of the core algorithms.

2. **Phase Two: Core Algorithm Development Week (Weeks 4-6):** Based on user needs, this phase lasts two weeks and focuses on implementing core functionalities. The emphasis is on developing a one-pass temporal BFS algorithm. We will ensure the algorithm can efficiently handle dynamic network data and recalculate node reachability in real time based on changes or deletions of edge weights in "What-if" scenarios. And eventually integrate the core BFS algorithm with a visualization engine (such as Streamlit).

3. **Phase three: System Testing and Optimization Week (Weeks 6-8):** We will test and evaluate the project product with suitable datasets and record the outputs.

4. **Phase four: Demonstration and Final Summary (Weeks 9):** Our team will make final improvements in the feedback phase and prepare demonstration materials.

3.4 Personnel and Task Dependencies

3.4.1 Personnel

Division and Time Arrangement (Who, What, When)

This project is structured into three distinct stages to balance collective foundational work with specialized technical development.

Phase 1: Foundation (Pre-Feb 23) Before February 23, the team operated under a unified management model. Weekly tasks were decided collectively to ensure all members gained a comprehensive understanding of the project objectives. This collaborative effort was reflected in the CA1 submission, where every member contributed to the initial research and project scoping.

Phase 2: Specialized Development (Feb 23 - April 1) During the core development period, the team transitions into specialized roles to maximize efficiency:

- **UI Design and Requirement Analysis (Tang Zixuan):** Leads the preliminary research and layout planning.
- **Data Cleaning (Ranjan Ankit, Zeng Wenbin):** Responsible for defining temporal network standards and preprocessing raw data.
- **BFS Algorithm (Ce Wang, Li Bowen):** Focuses on the core time-aware algorithm logic and path generation.
- **Web Implementation (Liu Yiqing, Xu Xiaohan):** Builds the interactive frontend framework based on Streamlit.

Phase 3: Integration and Finalization (April 1 - Post) After the delivery on April 1st, the team will reconvene to handle bug fixes and system optimization through shared task assignments. The submission of CA2 will be the result of a joint effort. For the final report (CA4), each member will write their own chapters, documenting their specific technical findings and implementation effects to ensure a comprehensive project narrative. In addition, a separate Individual Contribution report will be provided.

3.4.2 Task Dependencies

All links of the project are closely interconnected. Firstly, the UI group needs to determine the interaction requirements, which directly determine which fields the data group needs to clean and the container layout of the web group. Secondly, the standard adjacency list output by the data group is a prerequisite for the algorithm group to operate; if the data format is determined late, the algorithm logic cannot be implemented. Finally, the web group relies on the function interfaces provided by the algorithm group for dynamic rendering.

The overall process presents a linear dependency:

$$\text{requirement definition} \rightarrow \text{data processing} \rightarrow \text{algorithm implementation} \rightarrow \text{front-end integration.}$$

Any delay in the preceding links will cause subsequent development to enter an "idle running" state.

3.4.3 Implementation Plan

Implementation flow chart:

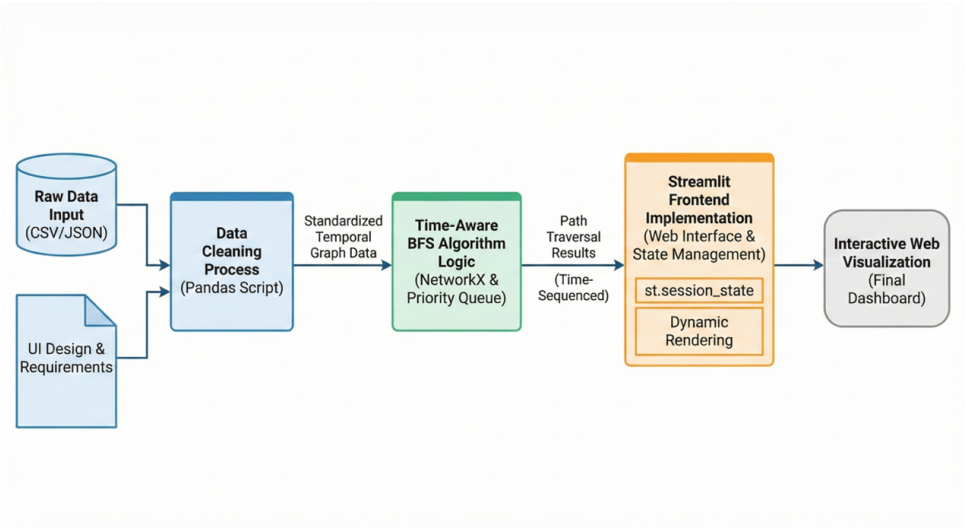

Figure 1:

3.4.4 Gantt Chart and Risk Assessment and Mitigation Measures

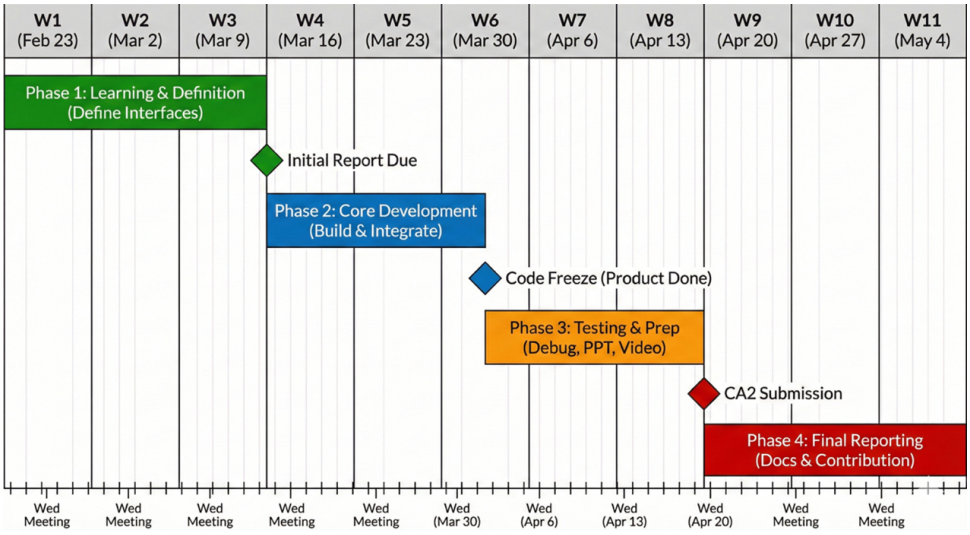

Figure 2:

4 Risk Management

4.1 Technical Implementation Risk

Since the "time-aware BFS" is different from traditional algorithms and involves logical judgment of time steps, there may be a risk that the high algorithm complexity leads to slow web page response.

Mitigation Measures: The algorithm group must complete the feasibility prototype verification by March 11. If the efficiency is low, consider preprocessing the data or simplifying the time window.

4.2 Interface Docking Risk

Inconsistent data formats or changes in function entry parameters may cause the web terminal to fail to call the algorithm normally.

Mitigation Measures: Strictly implement the Wednesday meeting system, requiring each group to submit a written "Interface Protocol Document" by March 11, and all subsequent changes must be known to all members.

4.3 Time Progress Risk

The development cycle is only three weeks, and the later tasks of PPT and report are heavy, which may lead to low-quality output due to rushing work.

Mitigation Measures: Strictly take turns to record meeting minutes in alphabetical order, clarify the weekly Wednesday deadline goals, break down major tasks into weekly tasks, and ensure that bug fixes are scheduled in the buffer period at the beginning of April.

5 Resources, Data, & Tools

5.1 Data Requirements and Ethical Approval

The dataset currently used comes from the Math Overflow temporal network in the Temporal Network category of the Stanford Large Network Dataset Collection. If a more suitable temporal network dataset is identified during the course of the project, the team will consider replacing the dataset to ensure the validity and effectiveness of the research analysis.

This dataset contains three columns parameters, SRC, TGT and Unix.

SRC: the source node(user)'s ID

TGT: the target node(user)'s ID

Unix: Unix timestamp

Since the dataset is publicly available and all users are represented by IDs, without real names or personally identifiable information included, ethical approval is not required.

(NOTE: To Ranjan Ankit: We don't actually have to use this dataset for the whole project; I'm just mentioning it here because it's relevant to the part I'm covering in CA1.)

5.2 Team Resources and Skills Deployment

All teammates have independent Python programming skills. The development tasks about this project is mainly divided into four parts: Data Cleaning(2 members), BFS Algorithm(2 members), Web Implementation(2 members), UI Design and Requirement Analysis(1 member). To maximize each member's professional strengths, the tasks were allocated according to personal skills and interests, which make members focus on the areas which their excel, improving the development efficiency and quality of project.

5.3 Software Development and Execution Environment

The software for this project is mainly developed on the team members' personal computers, using Python as major programming language. Data Cleaning, BFS Algorithm, Web Implementation, UI Design and Requirement Analysis all develop on Pycharm.

The software will be deployed as a Streamlit interactive web application, which allows user visualise patterns in temporal network(temporal motifs).

5.4 Project Tools and Their Applications

For this project, the main Python libraries used are pandas, Streamlit, and NetworkX.

Pandas: which can load data from various file format, including: CSV, JSON, SQL and Microsoft Excel. It also provides functionality operations such as merging datasets, selecting features, data cleaning and features processing.

Streamlit: using to quickly generate and share identify interactive web application. The core concept of Streamlit is let user can use only Python code(without front-end knowledge) to build professional data application.

NetworkX: a library for the creation, manipulation, and analysis of complex networks. It provides simply method of create node and edge, as well as directly way to visit network. NetworkX supports various kinds of networks, including undirected graph, directed graph, multigraph[3].

References

[1] Kempe, D., Kleinberg, J. and Kumar, A. (2002) 'Connectivity and inference problems for temporal networks'. *Journal of Computer and System Sciences*, 64(4), pp. 820–842.

[2] Holme, P. and Saramäki, J. (2012) 'Temporal networks'. *Physics Reports*, 519(3), pp. 97–125. `https://doi.org/10.1016/j.physrep.2012.03.001`

[3] Hagberg, A., Swart, P. and S Chult, D. (2008) 'Exploring network structure, dynamics, and function using NetworkX'. *Proceedings of the 7th Python in Science Conference*, pp. 11–15.